

نظام إدارة مشاريع التخرج (PMS)

الإضافتان المقترحتان لتحسين محرك الجدولة

شرح مفاهيمي للفهم

التهيئة الذكية • إصلاح التعارضات

دليل تفسيري — لماذا نحتاجهما، كيف يعملان، وماذا يتغيّر عملياً

يوليو 2026

مكمل لدليل الإصدار 2.0 • تحسينات مقترحة (لم تُنفذ بعد)

الملخص

بعد تطبيق الإصدار 2.0 (اللياقة بالعقوبة، الاختيار المعجمي، لجان جاهزة)، أصبح محرك الجدولة قابلاً للاستخدام الفعلي في حجم 15-30 مشروعاً. لكن عند توسّع العدد أو ضيق الموارد، يظهر عنق زجاجة: المحرك يبدأ من جداول عشوائية بالكامل، ويكتشف التعارضات بعد التوليد لا أثناءه. يقترح هذا الدليل إضافتين مكملتين: (1) التهيئة الذكية — بدء التشغيل بجداول شبه صالحة بدل العشوائية الكاملة؛ (2) إصلاح التعارضات — خطوة تصحيح فورية بعد كل تراوج وطفرة. يشرح الدليل الفكرتين بأمثلة عملية، مقارنة قبل وبعد، وكيف تتكاملان مع ما هو موجود اليوم — دون تفاصيل برمجية.

الكلمات المفتاحية: تهيئة ذكية، إصلاح التعارضات، خوارزمية جينية، تحسين الجدولة، الإصدار 2.0.

فهرس المحتويات

1. لماذا هاتان الإضافتان؟

2. كيف يعمل المحرك اليوم (v2) — نقطة البداية

3. الإضافة الأولى: التهيئة الذكية — الفكرة

4. الإضافة الأولى: مثال عملي خطوة بخطوة

5. الإضافة الثانية: إصلاح التعارضات — الفكرة

6. الإضافة الثانية: مثال عملي خطوة بخطوة

7. كيف تعمل الإضافتان معاً

8. مقارنة شاملة: قبل الإضافتين وبعدهما

9. متى نحتاجهما ومتى يكفي الوضع الحالي

10. الخلاصة

1. لماذا هاتان الإضافتان؟

الإصدار 2.0 حل مشكلة التقييم والترتيب: المدير يرى مقترحات مُرتَّبة ويمكنه اعتماد الأفضل. لكنه لم يحل مشكلتين في التوليد:

- البداية العشوائية: كل تشغيل يبدأ بمئة جدول عشوائي — كأنك ترمي 100 ورقة على الطاولة وتنتظر أن تصلح نفسها.
- اكتشاف متأخر للتعارض: عند تغيير وقت أو قاعة، لا يُتحقق فوراً من تعارض القاعة أو الأستاذ — يُكتشف لاحقاً عند التقييم.

تنبيه: هاتان الإضافتان مقترحتان لتحسين المستقبل — لم تُنفَّذا بعد في النظام. هذا الدليل للفهم والتخطيط، وليس وصفاً لسلوك الشاشة الحالية.

Problem today	Proposed addition	Benefit
100% random start	Smart initialization	Faster path to valid schedules
Conflicts found only at evaluation	Repair operator	Fewer violations per generation

Table 1: Two gaps and their proposed solutions

2. كيف يعمل المحرك اليوم (v2) — نقطة البداية

لفهم الإضافتين، نحتاج صورة واضحة لما يحدث الآن عند ضغط «تشغيل الخوارزمية»:

1. يُبنى سياق الجدولة (مشاريع، قاعات، لجان، فتحات زمنية).
2. يُنشأ 100 جدول عشوائي — كل جدول = جلسة لكل مشروع.
3. تُقيَّم الجداول (مخالفات صارمة + درجة ناعمة).
4. تتكرر 50 جيلًا: اختيار، تزاوج، طفرة، استبدال الأضعف.
5. يُختار أفضل 3 جداول متميزة ويُعرضون للمدير.

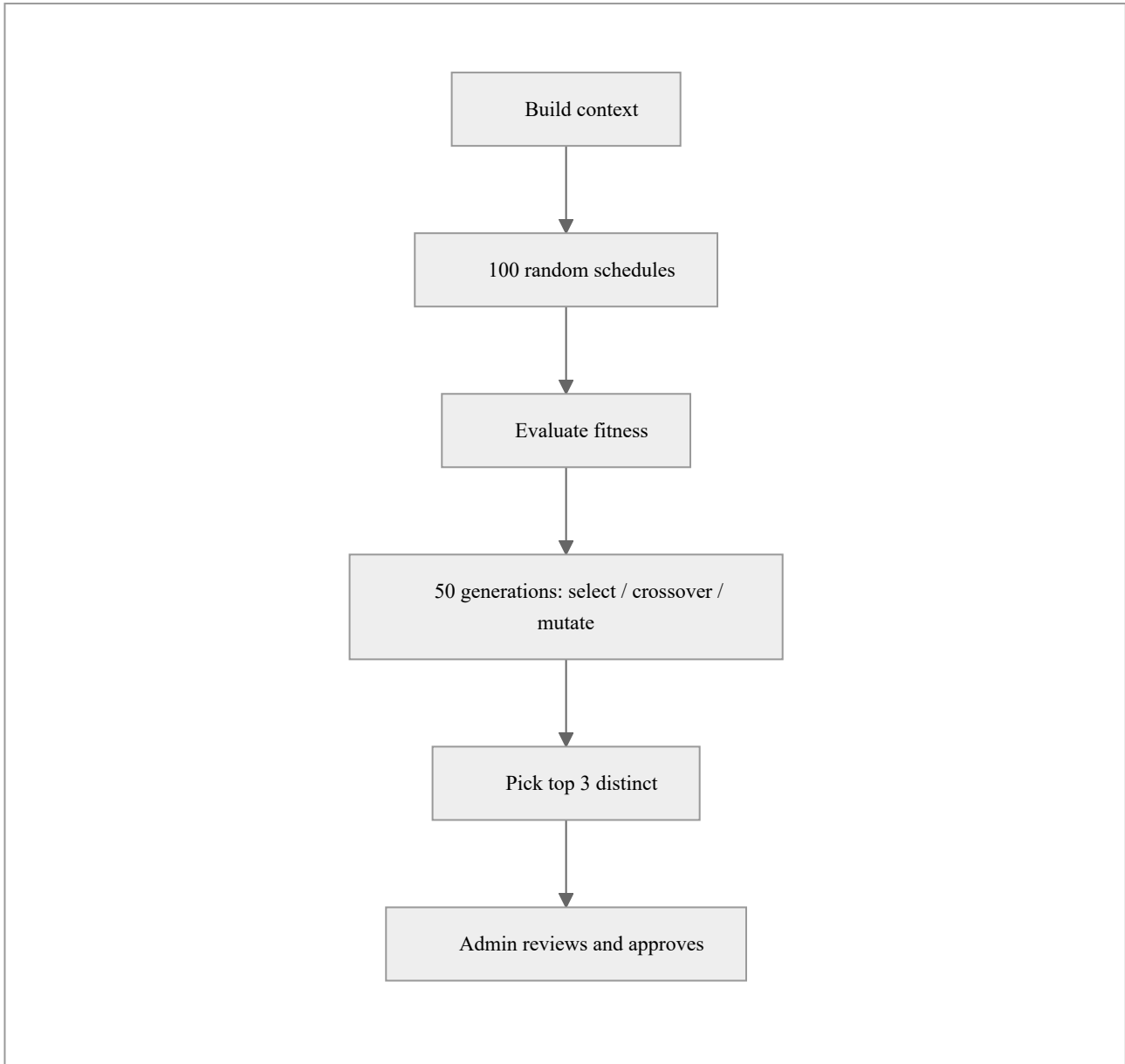


Figure 1: Current v2 engine flow (baseline)

أين الضعف؟ في الخطوتين الأولى والرابعة: البداية عشوائية بالكامل، والطفرة قد تُنشئ تعارضاً جديداً دون إصلاحه فوراً.

3. الإضافة الأولى: التهيئة الذكية — الفكرة

تعريف: التهيئة الذكية (Smart Initialization)

بدلاً من إنشاء كل الجداول الأولية عشوائياً، يُنشئ المحرك عدداً من الجداول «شبه الصالحة» بمنطق بسيط: يوزع المشاريع واحداً تلو الآخر على أول فتحة زمنية وقاعة متاحة دون تعارض — ثم يُكمل باقي المجتمع عشوائياً أو بمتغيرات طفيفة

3.1 تشبيه مبسط

تخيّل أنك ترتّب 15 اجتماعاً في تقويم:

- الطريقة العشوائية (اليوم): تضع كل اجتماع في وقت عشوائي — كثير من التعارضات، تحتاج ساعات لتصحيح.
- الطريقة الذكية (المقترحة): تبدأ بالاجتماع الأول في أول فتحة فارغة، الثاني في أول فتحة لا تتعارض، وهكذا — تحصل على مسودة جيدة من البداية.

3.2 ماذا يتغيّر في المجتمع الأولي؟

Population slice	Current (v2)	With smart init (proposed)
~20 schedules	Random	Greedy — one conflict-aware pass
~30 schedules	Random	Mutations of the greedy seed (small changes)
~50 schedules	Random	Fully random (diversity preserved)

Table 2: Proposed population mix (100 total)

المهدف ليس إلغاء العشوائية — الخوارزمية الجينية تحتاج تنوعاً — بل زرع بذور جيدة بين العشوائي حتى لا يبدأ التطور من الصفر

كل مرة.

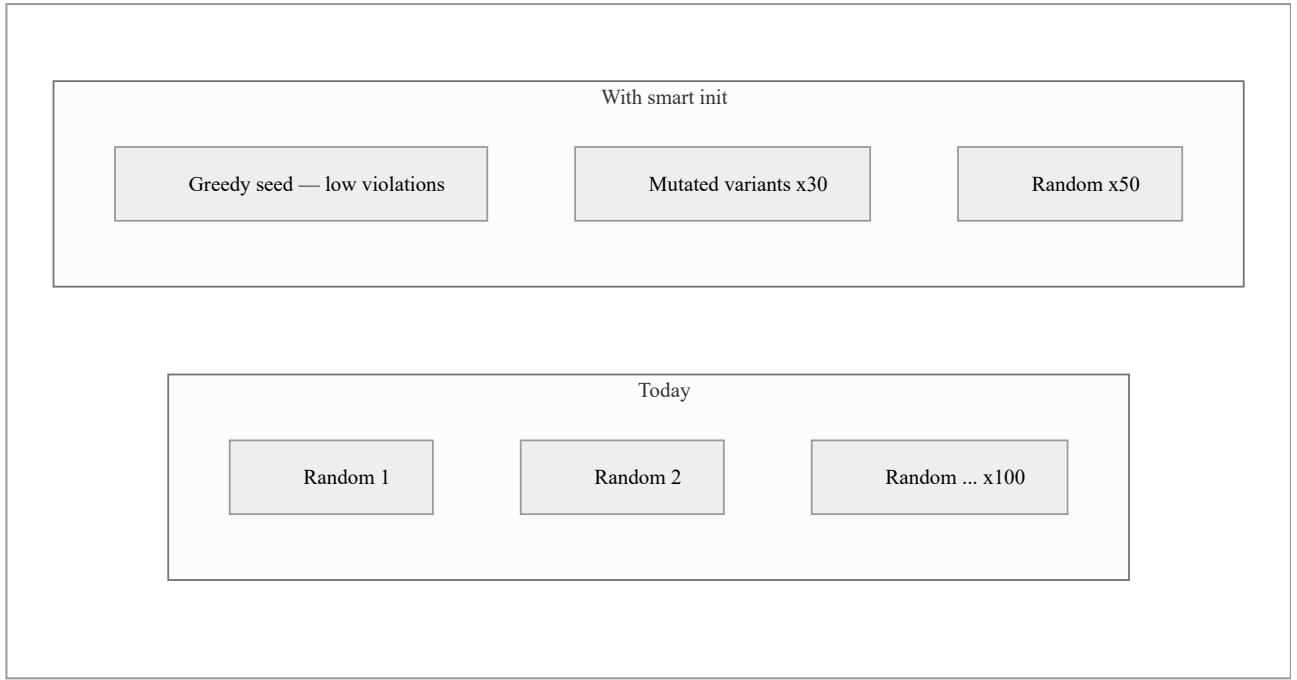


Figure 2: Random-only vs mixed initialization

4. الإضافة الأولى: مثال عملي خطوة بخطوة

السياق: 5 مشاريع (لتبسيط الشرح)، قاعتان (A و B)، فتحان يوم السبت: 10:00 و 11:00، لجنة واحدة جاهزة.

4.1 التهيئة العشوائية (اليوم)

Project	Random slot	Random room	Conflict?
P1	Sat 10:00	A	—
P2	Sat 10:00	A	Room A double-booked
P3	Sat 10:00	B	Committee double-booked
P4	Sat 11:00	A	—
P5	Sat 10:00	B	Committee double-booked

Table 3: Random init — 3 hard conflicts from the start

4.2 التهيئة الذكية (المقترحة)

Step	Project	Chosen slot	Chosen room	Reason
1	P1	Sat 10:00	A	First free slot
2	P2	Sat 10:00	B	Room A taken; committee free at 10:00 in B
3	P3	Sat 11:00	A	Committee busy at 10:00; next slot
4	P4	Sat 11:00	B	Only remaining free combo
5	P5	—	—	No slot left → flagged early

Table 4: Greedy init — 0 conflicts among schedulable projects; capacity gap visible

الفائدة العملية: الجيل الأول يبدأ بجدول فيه 0-1 مخالفة بدل 5-10. المحرك يصل أسرع إلى مقترحات قابلة للاعتماد — خاصة مع +40 مشروعاً.

5. الإضافة الثانية: إصلاح التعارضات — الفكرة

تعريف: إصلاح التعارضات (Repair Operator)
خطوة تصحيح تُنفَّذ بعد التزاوج والطفرة مباشرة: تفحص الجدول الجديد وتُصلح التعارضات البسيطة تلقائياً — نقل جلسة لقاعة فارغة، أو لأقرب فتحة زمنية لا يتعارض فيها الأستاذ أو القاعة — قبل إعادة التقييم.

5.1 لماذا نحتاجها رغم وجود التقييم؟

التقييم (اللياقة بالعقوبة) يُجبرنا أي جدول أفضل — لكنه لا يُصلح الجدول. إصلاح التعارضات يحوّل جدولاً «سيئاً» إلى جدول «مقبول» في نفس الجيل، بدل انتظار 10 أجيال حتى تصل الطفرة العشوائية للحل.

5.2 قواعد الإصلاح المقترحة (بالترتيب)

1. **تعارض قاعة:** نفس القاعة، نفس الوقت لمشروعين → انقل أحدهما لقاعة فارغة في نفس الوقت.
2. **تعارض أستاذ:** نفس عضو اللجنة في جلستين بنفس الوقت → انقل إحداها لأقرب فتحة يكون فيها الأستاذ والقاعة متاحين.
3. **تعارض مشرف:** مشرفان لمشروعين في نفس الوقت (حضور مناقشتين) → انقل إحداها (قيد مقترح إضافي).
4. **إن فشل الإصلاح:** يبقى التعارض — والتقييم يعاقبه كالعادة.

Stage	Without repair	With repair
After crossover	Child may inherit 2 room conflicts	Repair moves 2 sessions → 0 conflicts
After mutation	New random room may double-book	Repair picks next free room
Evaluation	Penalty applied	Often no penalty needed

Table 5: Repair operator effect per generation step

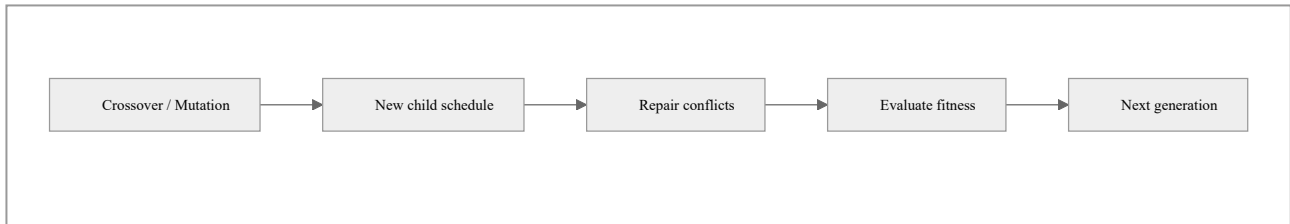


Figure 3: Where repair fits in the evolution loop

6. الإضافة الثانية: مثال عملي خطوة بخطوة

الموقف: بعد تزاوج جدولين، حصل التعارض التالي:

Project	Time	Room	Committee member Dr. Samer
P2	Sat 10:00	A	Yes
P7	Sat 10:00	B	Yes — CONFLICT
P9	Sat 11:00	A	No

Table 6: Conflict after crossover — Dr. Samer at two sessions same time

6.1 بدون إصلاح (اليوم)

الجدول يُقيّم → مخالفة صارمة واحدة على P7 → درجة منخفضة → يُقارَن بجدول أخرى → قد يُستبدل في الجيل التالي إن وُجد أفضل. قد يحتاج عدة أجيال للتخلص من التعارض.

6.2 مع إصلاح التعارضات (المقترح)

1. يكشف الإصلاح: د. سامر محجوز السبت 10:00 في P2 و P7.
2. يبحث عن أقرب فتحة لـ P7: السبت 11:00 — د. سامر متاح، القاعة B فارغة.
3. ينقل P7 إلى السبت 11:00، القاعة B.
4. يُقيّم الجدول → 0 مخالفات صارمة.

Project	After repair — Time	Room	Violations
P2	Sat 10:00	A	0
P7	Sat 11:00	B	0 — fixed
P9	Sat 11:00	A	0

Table 7: Same schedule after repair — conflict resolved in one step

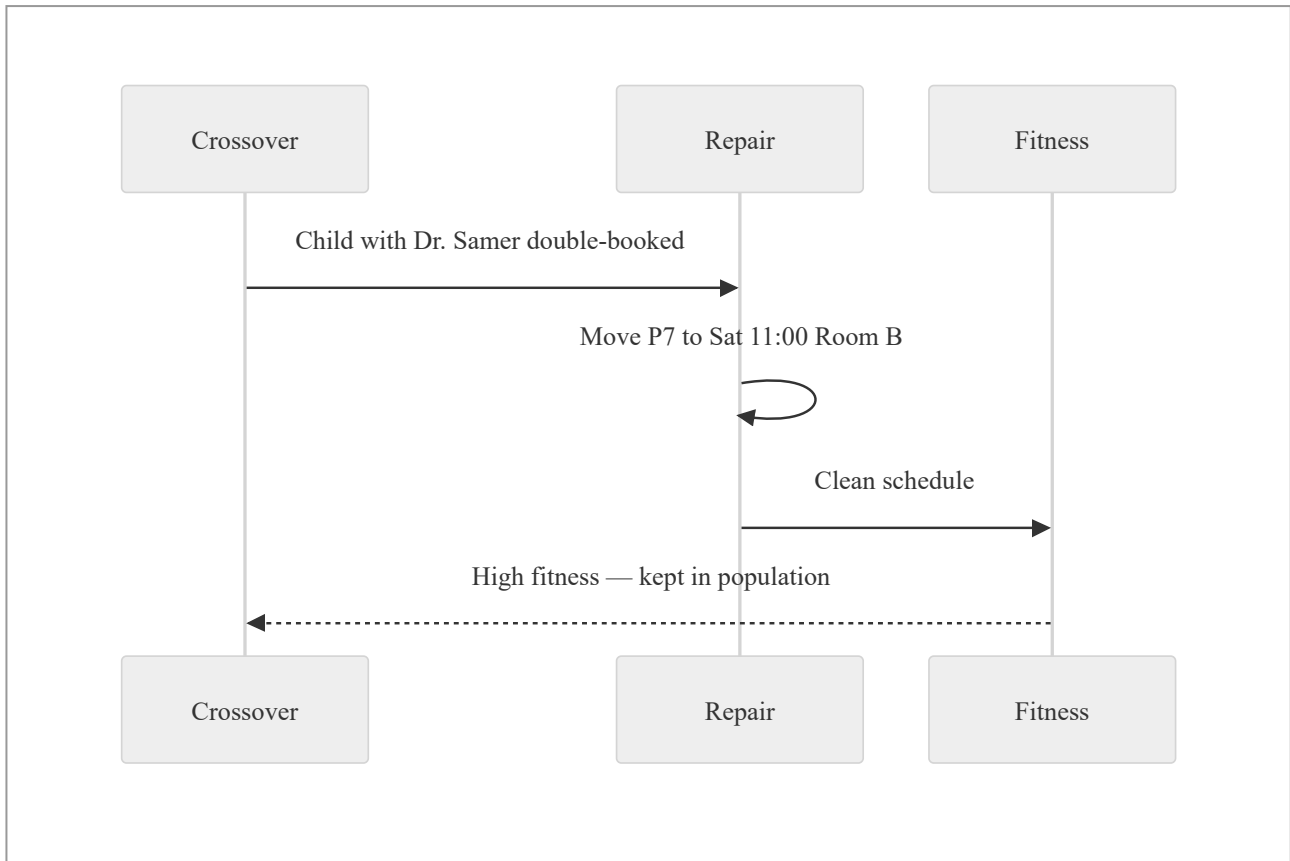


Figure 4: Repair example — one conflict fixed before evaluation

7. كيف تعمل الإضافتان معاً

الإضافتان مكملتان لا منافستان:

Addition	When it acts	What it solves
Smart initialization	Once — at engine start	Bad starting point — too many random conflicts
Repair operator	Every generation — after each child	New conflicts introduced by crossover / mutation

Table 8: Complementary roles of both additions

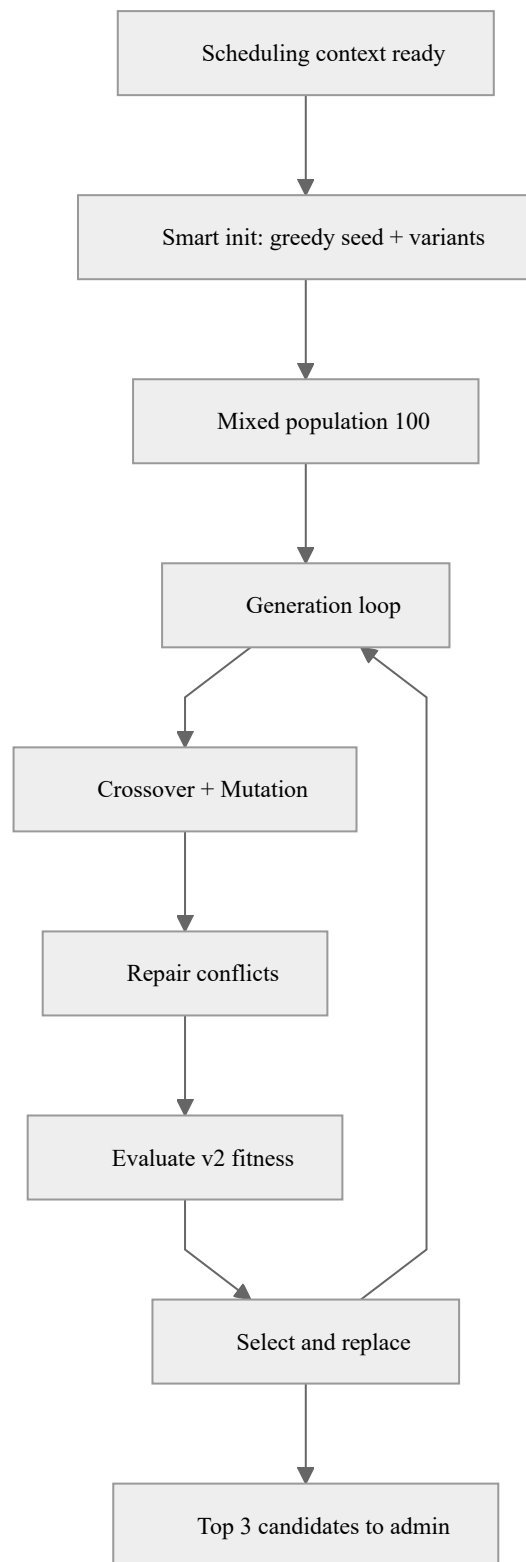


Figure 5: Full proposed engine with both additions

التكامل مع v2: اللياقة بالعقوبة والترتيب المعجمي يقيان كما هما — الإضافتان لا تستبدلهما، بل تُقلّلان عدد الجداول التي تحتاج العقوبة أصلاً. المدير يرى نفس الشاشة: 3 مقترحات — لكنها تصل أسرع وأنظف.

8. مقارنة شاملة: قبل الإضافتين وبعدهما

Metric	v2 today	v2 + both additions (proposed)
Typical conflicts in generation 1	High (random start)	Low (greedy seeds)
Generations to reach 0 violations	10–30 (varies)	3–10 (estimated)
Warning: no valid schedule	More likely at 40+ projects	Less likely
Execution time (~30s cap)	Uses full budget	May finish earlier (with early stop)
Admin experience	Same 3 candidates UI	Same UI — higher quality candidates
Implementation risk	—	Low — additive, no rewrite

Table 9: Before/after comparison — v2 vs v2 + proposed additions

8.1 مثال مجمّع: 30 مشروع سيمينار

Scenario	v2 today	With both additions
Best candidate violations	Often 0; sometimes 1–2	Usually 0
Candidate #2 quality	May be similar to #1	More likely distinct and valid
Admin decision time	Review + maybe re-run	Review + approve first try

Table 10: 30-project seminar — expected practical difference

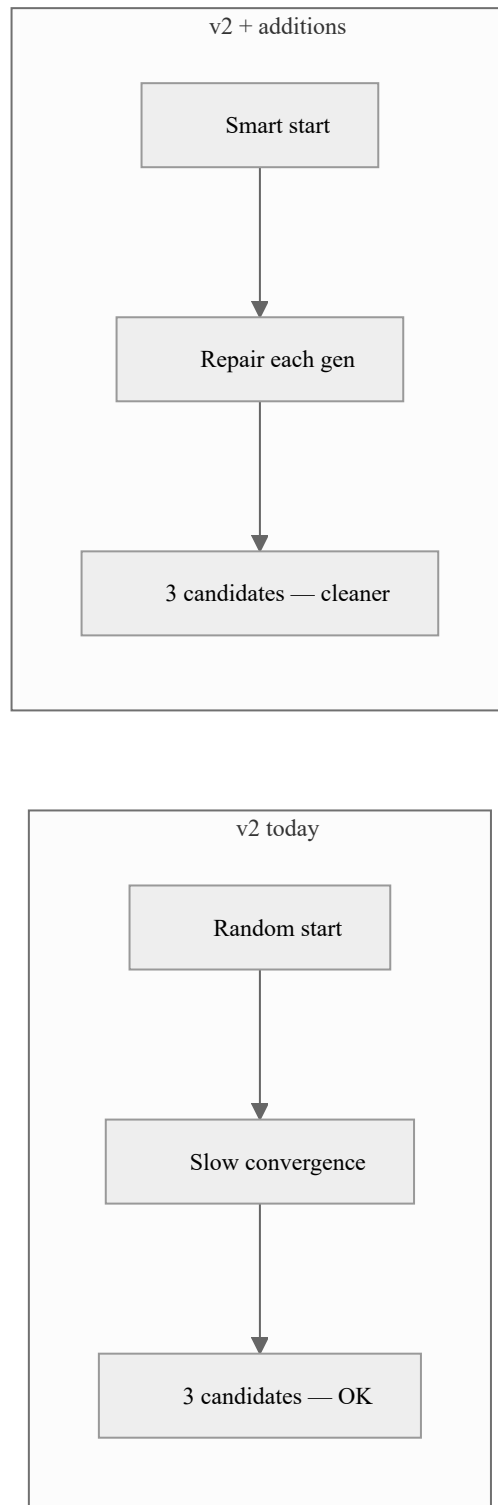


Figure 6: Overall improvement path

9. متى نحتاجهما ومتى يكفي الوضع الحالي

Situation	Recommendation
15–30 projects, 3 committees, 8 rooms	v2 today is sufficient — no urgent need
40+ projects or tight room capacity	Smart init + repair recommended
Frequent warning: no valid schedule	Implement both additions
Admin re-runs engine multiple times	Smart init reduces re-runs
Final defense season (high stakes)	Both additions + early stop + capacity check

Table 11: When to implement the proposed additions

أولوية التنفيذ المقترحة: (1) إصلاح التعارضات — تأثير فوري في كل جيل؛ (2) التهيئة الذكية — تأثير على البداية. يمكن تنفيذهما معاً في مرحلة واحدة لأحدهما لا يتعارضان.

10. الخلاصة

الإضافة الأولى — التهيئة الذكية: تبدأ المحرك من جداول شبه صالحة بدل العشوائية الكاملة، كأنك تضع مسودة منطقية على الطاولة قبل التحسين. **الإضافة الثانية — إصلاح التعارضات:** تُصلح الأخطاء فور توليدها في كل جيل، بدل انتظار التطور العشوائي لإصلاحها. معاً، تُكَمِّلان الإصدار 2.0 دون استبداله: نفس واجهة المدير، نفس 3 مقترحات، لكن مساراً أقصر إلى جداول خالية من المخالفات. للحجم الحالي (15–30 مشروع) الوضع الحالي كافٍ؛ للتوسع أو المواسم الضيقة، هاتان الإضافتان هي التحسين الأنسب بأقل مخاطرة.